FIELD NOTES

GitHub API Field Guide

The GitHub API stops working quietly: a list that returns thirty items, a webhook failing for a month, a 403 with no Retry-After. Every script here is read only.

16 fixes, each one a written note with diagrams and a small Python and Node.js script that finds the problem and repairs it. All 16 have a script in the public repo.

Before you run anything. Every script starts in a dry run: it reports what it would change and writes nothing. Read that output first, then set `DRY_RUN=false` when you are satisfied it has understood your data.

Scripts: github.com/allanninal/github-api-fixes
Guides: allanninal.dev/github
All 14 repos: github.com/allanninal

The fixes

1 **a permission error is disguised as 404 Not Found**

The repository is open in a browser tab in front of you. The script asks for the same repository and gets 404 {"message":"Not Found"}. Somebody checks the spelling, then the owner name, then the case of both, then writes a ticket saying the API is broken. The API is not broken. It is refusing to tell you that the repository exists, because telling you would leak the existence of a private repository to a credential that has no business knowing about it.

`github_404_triage.py`
https://www.allanninal.dev/github/404-masking-403/
https://github.com/allanninal/github-api-fixes/tree/main/404-masking-403

---

2 **resource not accessible by integration on one endpoint**

Nineteen endpoints work. The twentieth returns 403 {"message":"Resource not accessible by integration"}, which names no permission, no resource and no level, and reads like a platform bug rather than a configuration one. It is the one failure in this cluster where GitHub actually tells you the answer: it is in the x-accepted-github-permissions response header on that very 403, which almost no HTTP client shows you by default.

`github_app_permission_diff.py`
https://www.allanninal.dev/github/app-permission-missing/
https://github.com/allanninal/github-api-fixes/tree/main/app-permission-missing

---

3 **the compare endpoint stops at 250 commits and says nothing**

The release-notes generator diffs v4.2.0...v4.3.0 and produces a changelog that looks entirely plausible. It has 250 entries. The release contains 812 commits, and the ones it dropped are not the boring ones at the end — the shape of what came back is not what the code assumed at all.

`github_compare_truncation.py`

https://www.allanninal.dev/github/compare-250-commit-cap/

https://github.com/allanninal/github-api-fixes/tree/main/compare-250-commit-cap

---

4 **the same webhook URL is registered on the org and the repo**

The bot comments twice on every pull request. Someone checks the receiver for a retry loop, finds none, and blames GitHub for sending duplicates. GitHub is sending exactly one copy of the event to each hook that asked for it — and two hooks asked, one on the repository and one on the organization, both pointing at the same URL.

`github_duplicate_hooks.py`

https://www.allanninal.dev/github/duplicate-webhooks/

https://github.com/allanninal/github-api-fixes/tree/main/duplicate-webhooks

---

5 **the installation covers only some repositories, silently**

The scanner reports clean. Every repository it looked at was fine, every check passed, and the summary at the bottom says so. It looked at twelve repositories. The organization has a hundred and forty. Nothing errored, nothing warned, and no line of that report is untrue — the App installation was set to selected repositories at some point in 2023 and the other hundred and twenty-eight have never been inside its field of view.

`github_app_coverage_audit.py`

https://www.allanninal.dev/github/installation-repository-selection-partial/

https://github.com/allanninal/github-api-fixes/tree/main/installation-repository-selection-partial

---

6 **only the first page is read because the Link header is ignored**

The request returned 200. The JSON is a well-formed array of pull requests. Your audit read it, counted 30, and reported that the repository is tidy. There are 340 open pull requests. Nothing failed, nothing was logged, and the number you are now acting on is wrong by an order of magnitude — the answer was complete for one page and the rest was advertised in a header nobody read.

`github_link_header_audit.py`

https://www.allanninal.dev/github/link-header-not-followed/

https://github.com/allanninal/github-api-fixes/tree/main/link-header-not-followed

---

7 **polling without ETags spends full quota on unchanged data**

The dashboard polls eight endpoints every thirty seconds and burns through 5,000 requests before lunch. Almost nothing it fetches has changed. GitHub has been sending an etag on every one of those responses, and every response that comes back 304 Not Modified is free — it does not count against the rate limit at all. This is the one problem in this section where the fix pays for itself in a number you can print.

`github_etag_saving.py`

https://www.allanninal.dev/github/no-conditional-requests/

https://github.com/allanninal/github-api-fixes/tree/main/no-conditional-requests

---

8 **per_page is unset so every list costs 3.3x more requests**

The job is correct. It follows rel="next" to the end, it reads every issue, and it burns through the hourly quota by lunchtime. Nothing is broken and nothing needs debugging — it is simply making three and a third times as many requests as it needs to, because per_page was never set and the default is 30.

`github_per_page_audit.py`

https://www.allanninal.dev/github/per-page-default-30/

https://github.com/allanninal/github-api-fixes/tree/main/per-page-default-30

---

9 **the client ignores retry-after and keeps hammering the API**

The log shows four hundred consecutive 403s, one second apart, for eleven minutes. The retry logic is working perfectly: it catches the error, waits, tries again, never gives up. GitHub said retry-after: 120 on the very first one, and the client threw that header away along with the rest of the response.

`github_backoff_plan.py`

https://www.allanninal.dev/github/retry-after-ignored/

https://github.com/allanninal/github-api-fixes/tree/main/retry-after-ignored

---

10 **org lists silently omit SSO-enforced organizations**

The user belongs to six organizations. GET /user/orgs returns four. Status 200, valid JSON, no errors array, nothing in the body that hints anything is absent. The two organizations that enforce SAML single sign-on for a token that was never authorized against them are simply not in the list, and the only place GitHub mentions it is a response header called X-GitHub-SSO.

`github_sso_partial_results.py`

https://www.allanninal.dev/github/saml-partial-results/

https://github.com/allanninal/github-api-fixes/tree/main/saml-partial-results

---

11 **search returns at most 1,000 results whatever total_count says**

The response says "total_count": 24831. You page through it at 100 per request, and on page 11 the API returns 422 Validation Failed with "Only the first 1000 search results are available". The count was true. The results were never there to fetch — total_count describes the match set, not the part of it you are allowed to page through.

`github_search_cap_audit.py`

https://www.allanninal.dev/github/search-1000-result-cap/

https://github.com/allanninal/github-api-fixes/tree/main/search-1000-result-cap

---

12 **over 100 concurrent requests trips a secondary rate limit**

Someone replaces a for loop with a Promise.all and the job goes from nine minutes to forty seconds, once. The next run returns 403 on two thirds of the requests. You check the quota, because a 403 from GitHub means the quota, and the quota says four thousand eight hundred requests left. Both numbers are true. They are about different limits.

`github_concurrency_probe.py`

https://www.allanninal.dev/github/secondary-limit-concurrency/

https://github.com/allanninal/github-api-fixes/tree/main/secondary-limit-concurrency

---

13 **bulk issue or comment creation exceeds 80 requests a minute**

The migration script imports 2,400 issues from the old tracker. It gets through about eighty of them, then every remaining call comes back 403. You check the quota and there are 4,900 requests left in it. The limit that stopped you is not counting requests. It is counting the things you created.

`github_content_burst_audit.py`

https://www.allanninal.dev/github/secondary-limit-content-creation/

https://github.com/allanninal/github-api-fixes/tree/main/secondary-limit-content-creation

---

14 **webhook deliveries are failing and nobody reads the log**

Someone opens a pull request and the bot that should comment on it says nothing. You grep your receiver's logs for the last hour and find no request at all, which points the finger at GitHub. It is not GitHub. The delivery happened, your server answered 502, and GitHub wrote that down in a log you have never opened.

`github_hook_delivery_audit.py`

https://www.allanninal.dev/github/webhook-deliveries-failing/

https://github.com/allanninal/github-api-fixes/tree/main/webhook-deliveries-failing

---

15 **the hook is not subscribed to the event you are waiting for**

The handler was written, reviewed, unit tested against a saved payload, and deployed. In production it has never executed once. There is no error anywhere because there is no failure anywhere: the hook was created years ago with push and pull_request, and the event your handler waits for has never been sent to it.

`github_hook_event_coverage.py`

https://www.allanninal.dev/github/webhook-event-not-subscribed/

https://github.com/allanninal/github-api-fixes/tree/main/webhook-event-not-subscribed

---

16 **a webhook with no secret sends no signature to verify**

The receiver has a signature check. It reads X-Hub-Signature-256, computes an HMAC over the body, compares in constant time, and returns 401 when they differ. It has never returned 401, because the hook it serves has no secret, so GitHub does not send the header, and the check quietly skips itself on every request.

`github_hook_secret_audit.py`

https://www.allanninal.dev/github/webhook-no-secret/

https://github.com/allanninal/github-api-fixes/tree/main/webhook-no-secret

---

MIT licensed. Free to use, change and ship. Written by Allan Niñal — allanninal.dev